

SmartTutor: Reliable Multi-Turn Homework Tutoring Agent

Group Members: Hu Nanchen, Hu Ruiqi, Chen Kaixuan

Presentation: No

1. What Our Agent Is Supposed to Do

SmartTutor is designed as a reliability-first homework tutor for math and history in multi-turn conversations: it should answer valid questions clearly, reject invalid or unsafe requests with stable reasons, carry context across short follow-ups, adapt explanation depth after users provide grade level (e.g., primary school or university year one), and summarize the conversation on request; the key success criterion is not being a general chatbot, but making consistent accept/reject decisions and maintaining coherent tutoring behavior across turns.

2. Agent Design and Routing Logic

Instead of a single prompt, we use a small pipeline with explicit responsibilities:

- orchestrator: single entry point and routing controller,
- conversation_manager: stores session history, grade, and last response type,
- triage_agent: classifies each turn into valid_math, valid_history, or invalid, and detects intents such as grade info, summary, and chit-chat.
- guardrail_agent: decides whether the question should be rejected,
- answer_generator: generates math/history answers and summaries,
- multi_model_client: selects configured model and handles fallback.

Decision flow (per user turn)

1. Save user message into session history.
2. Classify message (`valid\_math`, `valid\_history`, or `invalid`) and intent.
3. Repair short follow-ups using previous valid context when appropriate.
4. Handle special intents first: grade-info turns update stored grade, summary turns call structured summary generation, simple social turns return polite responses instead of hard rejection.
5. Apply guardrails for scope/safety checks.
6. If accepted, dispatch to math/history answer generation with grade-aware prompts and recent history context.
7. Save assistant response for future turns.

3. Reliability and Guardrails

SmartTutor is intentionally conservative. It only answers math/history homework and rejects:

- non-homework requests (e.g., travel advice),
- non-math/non-history topics,
- overly local or niche questions outside intended scope,
- unsafe/inappropriate prompts.

To improve reliability, we combine LLM reasoning with deterministic mechanisms:

- keyword-based fallback classification for obvious cases,
- contextual carry-over for short follow-ups (e.g., "And more?"),
- remembered grade level for answer adaptation,
- model fallback when task-specific model access fails.

4. Validation Strategy

We validate with both automated tests and manual multi-turn examples.

- **Automated tests** cover accepted/rejected questions, summary behavior, follow-up continuity, grade adaptation, polite chit-chat, API consistency, and fallback behavior.
- **Manual smoke test** verifies end-to-end multi-turn behavior in one session, matching assignment requirements.

5. Required Example Dialogs (Final Evidence)

We provide six representative examples below covering accepted, rejected, and summary.

	User Input	Expected Behavior	Observed Behavior
1	$x+1=2$	Accepted as math homework	Agent solves the equation step-by-step correctly
2	the concept of calculus	Agent identifies the user's primary school level and provides a simplified explanation using analogies	Agent adapts explanation depth after user provides grade level (grade-aware prompt)
3	Who was the first president of France?	Accepted as history homework	Agent answers with Louis-Napoléon Bonaparte and brief context
4	How do I get to London?	Rejected (not homework)	Agent refuses and explains it is outside math/history homework scope
5	Who was HKUST's first president?	Rejected (too local/niche)	Agent refuses as outside intended general history-homework scope
6	Summarize our conversation so far.	Provide conversation summary	Agent returns a concise structured summary of prior turns

6. How to Run

From the 'smarttutor' directory:

```
pip install -r requirements.txt
python -m ui.gradio_app
```

Open `http://127.0.0.1:7861`.

Set credentials in `.env` (at least one valid provider key, e.g., `HKUST_AZURE_API_KEY` or `OPENAI_API_KEY`).